



**UNIVERSIDAD DE CASTILLA-LA MANCHA
ESCUELA SUPERIOR DE INFORMÁTICA**

<SISTEMA DE TORNEO E-SPORTS VALORANT >

Autores

-
- David Calzado Olmo --> David.Calzado1@alu.uclm.es
 - Marcos Loro Carrasco--> Marcos.Loro@alu.uclm.es
 - Marcos Sobrino García-Villaraco--> Marcos.Sobrino1@alu.uclm.es
 - Hatim Ben Ayad--> Hatim.Benayad1@alu.uclm.es
 - Luis Dueñas Recuero--> Luis.Duenas1@alu.uclm.es
-



Asignatura: Ingeniería del Software I

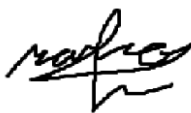
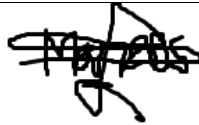
Grupo de Titulación (25/26):

Titulación: Grado en Ingeniería Informática

Fecha: 22/12/2025

Ficha del Trabajo:

Código:	A05	Fecha:	14/11/2025
Título:	SISTEMA DE TORNEO E-SPORTS VALORANT		

Equipo		Nº:
Apellidos y Nombre	Firma	Puntos
Calzado Olmo, David		20%
Carrasco Loro, Marcos		20%
Sobrino García-Villaraco, Marcos		20%
Ben Ayad, Hatim		20%
Dueñas Recuero, Luis		20%

Índice del proyecto

UNIVERSIDAD DE CASTILLA-LA MANCHA	1
1. Requisitos del Software	4
2. Modelo de Casos de Uso.....	5
A) Especificación Actores:	7
B) Por cada Caso de Uso: Información Básica, Detalles y Requisitos que satisface.	7
C) Para dos casos de uso (deberán decidirse junto con el profesor de teoría obligatoriamente): en la descripción de esos dos casos de uso, se detallarán el escenario principal y un escenario alternativo mediante los correspondientes flujos de eventos.....	9
3. Modelo de Análisis	11
3.1 Vista Estática.....	11
3.2 Vista dinámica.....	12
4. Modelo de Diseño	14
4.1 Vista dinámica.....	14
4.1.1 Diagramas de secuencia	14
4.1.2 Diagrama de estados.	15
4.2 Vista estática.	17
4.2.1. Diagrama de clases.....	17

1. Requisitos del Software

- **Restricciones explícitas del sistema:**

- A. Solo pueden registrarse estudiantes universitarios de la Universidad de Castilla La Mancha activos con matrícula válida y con su correo institucional.
- B. El sistema está exclusivamente diseñado para torneos de Valorant, no se admiten otros juegos.
- C. Cada usuario solo puede pertenecer a un equipo por torneo.

- **Especificación de Requisitos del Software:**

Administradores:

1. Los administradores podrán crear un torneo que generará una contraseña, esa contraseña será pedida al resto de jugadores para ingresar al torneo.
2. Los administradores pueden eliminar un torneo.
3. Los administradores del torneo podrán apuntar los resultados de las partidas. Estos resultados afectarán a la clasificación global del torneo.
4. Los administradores podrán registrar las infracciones cometidas por un usuario junto con el tipo de infracción (jugar de manera ilegítima, comportamiento antideportivo, etc). Estas infracciones serán apuntadas dentro de la información del jugador que las cometió. En caso de acumular 3 infracciones leves, o si el administrador cree que la infracción es suficientemente grave puede eliminar al jugador del torneo y sus credenciales serán bloqueadas de la BBDD, de forma que no pueda volver a participar en ningún torneo. Si es por infracciones leves, solo será sancionado una partida.
5. Los administradores pueden actualizar los datos del torneo.

Usuario genérico:

1. Todos los usuarios deben registrarse e iniciar sesión.

2. Cualquier usuario podrá ver el ranking del campus y la clasificación global obtenido por la BBDD.
3. Cualquier usuario podrá ver estadísticas de un equipo seleccionado.

Jugadores:

1. Los jugadores podrán crear equipos de 10 personas pasando a ser capitanes de dicho equipo.
2. Los jugadores podrán buscar equipos disponibles y solicitar unirse a uno.
3. Los jugadores podrán abandonar un equipo.

Capitanes:

1. Los capitanes pueden aceptar o rechazar la solicitud de unión de un jugador.
2. Los capitanes pueden disolver el equipo.
3. Los capitanes podrán gestionar la plantilla de su equipo, marcando cuales de los jugadores serán titulares en el siguiente partido. Deben estar marcados 5 en todo momento.

Base de datos ESI (Sistema Externo):

1. La base de datos proporciona los datos necesarios para la autenticación de los usuarios y la consulta de información, garantizando la coherencia y disponibilidad de los datos del sistema.

2. Modelo de Casos de Uso

• Diagrama de Casos de uso:

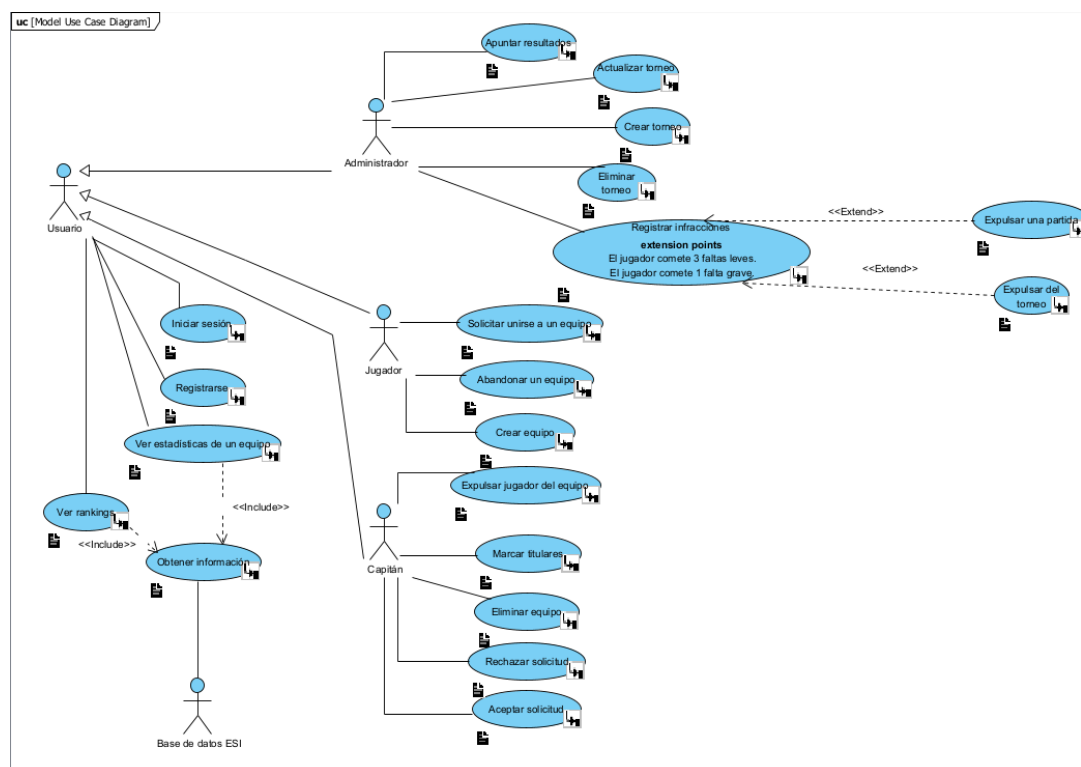
El sistema desarrollado permite la gestión de torneos universitarios de Valorant, estando restringido a estudiantes activos de la Universidad de Castilla-La Mancha que se autentican mediante su correo institucional. El sistema garantiza que cada usuario solo pueda pertenecer a un equipo por torneo.

Todos los usuarios deben registrarse e iniciar sesión para acceder a la aplicación. A través de los casos de uso de consulta, los usuarios pueden ver el ranking global y consultar las estadísticas de un equipo, funcionalidades que incluyen la obtención de información almacenada en la base de datos.

Los jugadores pueden crear equipos, solicitar unirse a uno existente o abandonarlo. Al crear un equipo, el jugador pasa a ser capitán, pudiendo aceptar o rechazar solicitudes, gestionar la plantilla marcando a los titulares y eliminar el equipo.

Los administradores pueden crear, actualizar y eliminar torneos, registrar los resultados de las partidas y gestionar infracciones. El registro de infracciones contempla escenarios alternativos mediante relaciones de extensión, como la expulsión temporal de un jugador o su expulsión definitiva del torneo, en función de la gravedad o acumulación de infracciones.

La base de datos ESI proporciona soporte a los distintos casos de uso, permitiendo la autenticación de usuarios y la consulta de información.



A) Especificación Actores:

- Usuario: Cualquier persona que quiera acceder al sistema, incluidos otros actores de este.
- Administrador: Son los que gestionan y supervisan el correcto funcionamiento del sistema. Crea, elimina o actualiza los torneos cuando es oportuno, y están atentos al comportamiento de jugadores.
- Jugador: Participa en el torneo. Si crea un equipo se convierte en capitán.
- Capitán: Misma función que jugador, pero gestiona su plantilla.
- Base de datos ESI: Proporciona información de los jugadores, como si han sido ya expulsados de algún torneo de la ESI.

B) Por cada Caso de Uso: Información Básica, Detalles y Requisitos que satisface.

- Administrador

1. Crear torneo
 - Descripción: Permite al administrador crear un torneo nuevo.
 - Requisito que satisface: Administrador - Requisito 1.
2. Eliminar torneo
 - Descripción: El administrador puede eliminar un torneo.
 - Requisito que satisface: Administrador - Requisito 2.
3. Apuntar resultados
 - Descripción: El administrador registra los resultados de las partidas.
 - Requisito que satisface: Administrador - Requisito 3.
4. Registrar infracciones
 - Descripción: El administrador registra las infracciones cometidas por los jugadores, especificando si son leves o graves.
 - Requisito que satisface: Administrador - Requisito 4.
5. Actualizar torneo
 - Descripción: Permite al administrador modificar o actualizar los datos de un torneo.
 - Requisito que satisface: Administrador - Requisito 5.
6. Expulsar una partida (*extends registrar infracciones*)
 - Descripción: Si un jugador acumula tres infracciones leves, se le expulsa de una partida.
 - Requisito que satisface: Administrador - Requisito 4.

7. Expulsar del torneo (*extends registrar infracciones*)

- Descripción: Si el jugador comete una falta grave, es eliminado del torneo y bloqueado en la base de datos.
- Requisito que satisface: Administrador - Requisito 4.

- Usuario

1. Registrarse

- Descripción: Permite a los estudiantes de la UCLM crear una cuenta con su correo institucional.
- Requisito que satisface: Usuario - Requisito 1.

2. Iniciar sesión

- Descripción: Permite a los usuarios autenticarse para acceder a las funciones del sistema.
- Requisito que satisface: Usuario - Requisito 1.

3. Ver rankings

- Descripción: Permite consultar la clasificación del campus y el ranking global.
- Requisito que satisface: Usuario - Requisito 2.

4. Ver estadísticas de un equipo

- Descripción: Permite consultar los datos y estadísticas de un equipo concreto.
- Requisito que satisface: Usuario - Requisito 3.

5. Obtener información (Base de datos ESI)

- Descripción: El sistema obtiene la información solicitada del equipo o ranking desde la base de datos.
- Requisito que satisface: Usuario - Requisitos 2 y 3.

- Jugador:

1. Solicitar unirse a un equipo

- Descripción: Un jugador puede enviar una solicitud para unirse a un equipo disponible.
- Requisito que satisface: Jugador - Requisito 2.

2. Abandonar un equipo

- Descripción: Permite a un jugador dejar su equipo.
- Requisito que satisface: Jugador - Requisito 3.

3. Crear equipo

- Descripción: Permite a un jugador crear un equipo convirtiéndose en capitán.
- Requisito que satisface: Jugador - Requisito 1.

- Capitán:

1. Eliminar equipo

- Descripción: El capitán puede disolver su equipo.
- Requisito que satisface: Capitán - Requisito 2.

2. Aceptar solicitud

- Descripción: El capitán acepta la solicitud de un jugador para que el jugador se una al equipo.
- Requisito que satisface: Capitán - Requisito 1.

3. Rechazar solicitud

- Descripción: El capitán rechaza la solicitud de un jugador que desea unirse al equipo.
- Requisito que satisface: Capitán - Requisito 1.

4. Expulsar jugador del equipo

- Descripción: El capitán puede expulsar a un jugador de su equipo.
- Requisito que satisface: Capitán - Requisito 3.

5. Marcar titulares

- Descripción: El capitán selecciona a los 5 jugadores titulares para el siguiente partido.
- Requisito que satisface: Capitán - Requisito 3.

C) Para dos casos de uso (deberán decidirse junto con el profesor de teoría obligatoriamente): en la descripción de esos dos casos de uso, se detallarán el escenario principal y un escenario alternativo mediante los correspondientes flujos de eventos.

Se han seleccionado dos casos de uso en los que explicaremos para cada uno su flujo de eventos que se da cuando el caso de uso se desarrolla con normalidad y uno de los posibles flujos de eventos que pueden llevarse a cabo si se cumplen determinadas condiciones que se desvían del desarrollo normal del caso de uso denominados los casos alternativos.

Caso de uso 1: Registrar infracciones

Actor principal: Administrador

◆ Escenario principal

1. Selecciona el jugador al que desea registrar una infracción.
2. Registrar infracción.
3. Actualizar registro de infracciones del jugador.
4. Devolver registro de infracciones del jugador.
5. Mostrar registro de infracciones.

Resultado: La infracción queda guardada en el registro del jugador.

◆ Escenario alternativo: Jugador no encontrado.

1. Selecciona el jugador al que registrar una infracción
2. Registrar infracción al jugador
3. Mostrar error de jugador no encontrado

Resultado: El jugador no ha podido ser encontrado, vuelve a intentarlo.

Caso de uso 2: Solicitar unirse a un equipo

Actor principal: Jugador

◆ Escenario principal

1. Elegir equipo al que desea mandar una solicitud
2. Mandar solicitud al equipo.
3. Comprobar hueco en equipo.
4. Comprobar previas solicitudes.
5. No hay solicitudes previas a este equipo.
6. Hay hueco suficiente en el equipo.
7. Mostrar estado de solicitud en espera

Resultado: La solicitud queda registrada y el capitán podrá aceptarla o rechazarla.

◆ Escenario alternativo: El equipo está completo

1. Elegir equipo al que desea mandar una solicitud
2. Mandar solicitud al equipo.
3. Comprobar hueco en equipo.
4. Comprobar previas solicitudes.
5. No hay solicitudes previas a este equipo.
6. El equipo está completo
7. Mostrar error de equipo completo

Resultado: La solicitud no se envía y el jugador sigue igual (sin equipo o en su equipo actual)

3. Modelo de Análisis

Contenido de la sección:

3.1 Vista Estática

- Desarrollo de la **Vista estática** mediante los **Diagramas de Clases de Análisis**

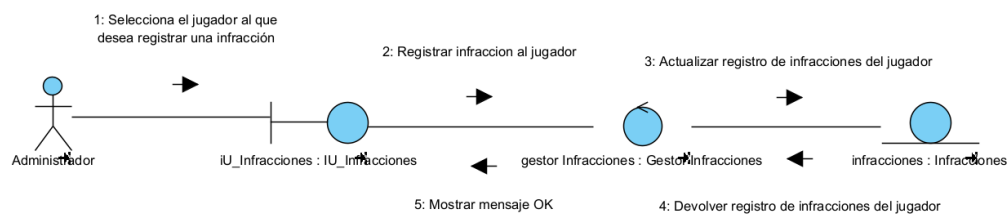
Diagrama de clases de análisis de vista estática global:



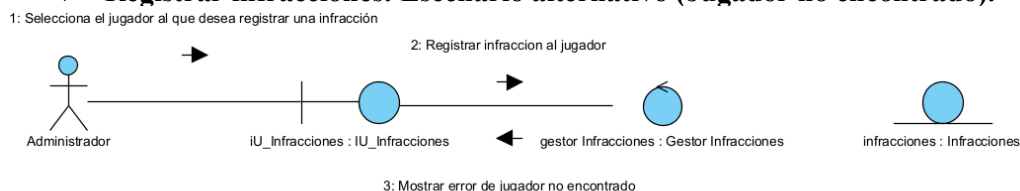
3.2 Vista dinámica

- Desarrollo de la **Vista dinámica** mediante los **Diagramas de Interacción de Análisis**

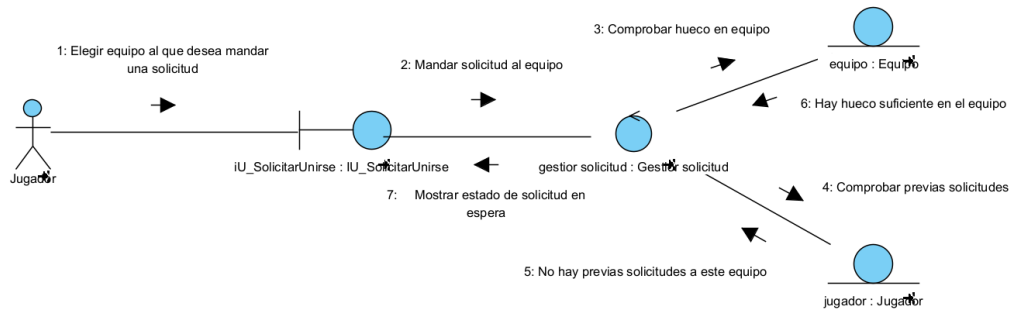
+ **Registrar infracciones. Escenario principal:**



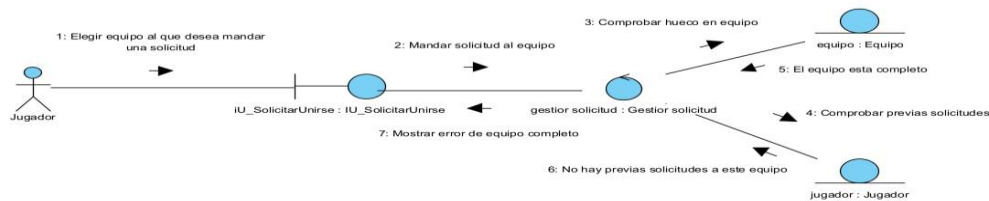
+ **Registrar infracciones: Escenario alternativo (Jugador no encontrado).**



- **Solicitar unirse a equipo. Escenario principal:**



Solicitar unirse a equipo: Escenario alternativo (El equipo está completo).



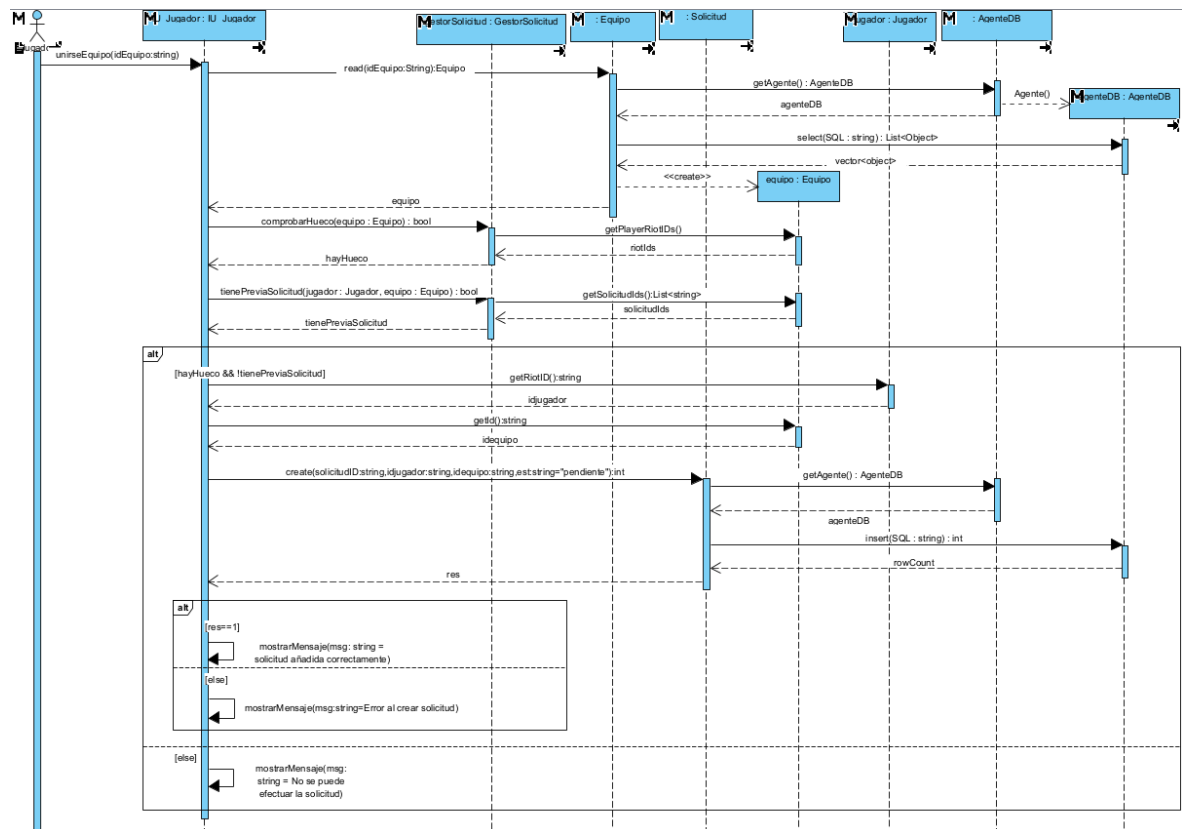
4. Modelo de Diseño

4.1 Vista dinámica

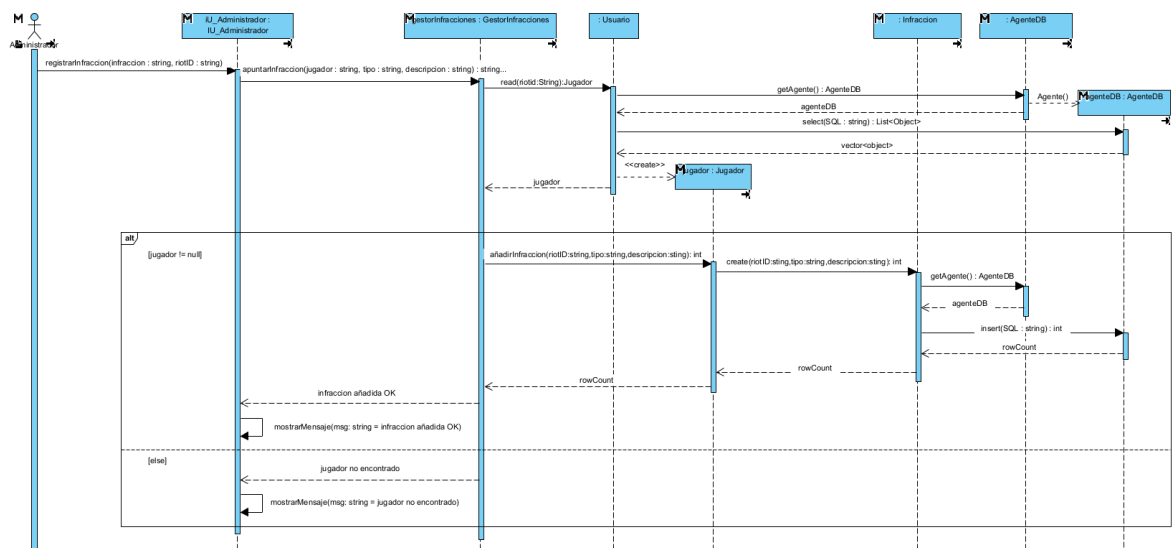
En el siguiente diagrama de secuencia se representa el comportamiento dinámico del sistema para nuestros 2 casos de uso elegidos. En él se muestran las interacciones entre el actor y las distintas capas del sistema (presentación, dominio y persistencia) detallando el orden en el que se realizan las llamadas a métodos y el intercambio de información necesario para llevar a cabo la funcionalidad descrita. Este diagrama permite comprender cómo se coordina el sistema internamente para dar respuesta a la petición del usuario.

4.1.1 Diagramas de secuencia

- **Caso de Uso: Solicitar unirse a un equipo**

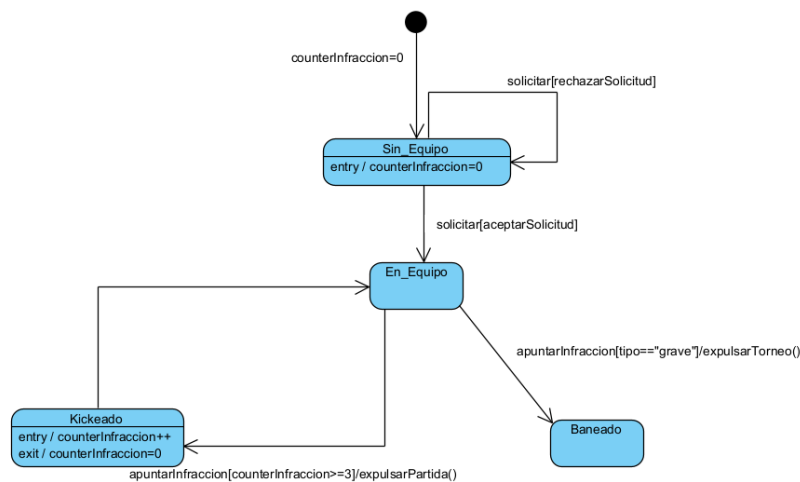


• Caso de Uso: Registrar infracciones



4.1.2 Diagrama de estados.

El diagrama de estados de la clase Jugador muestra la evolución de un jugador en el sistema, desde el estado Sin_Equipo hasta En_Equipo, y los posibles estados derivados de la gestión de infracciones. En función de la gravedad o acumulación de infracciones, el jugador puede ser sancionado temporalmente o expulsado definitivamente del torneo, reflejando las reglas definidas en los requisitos del sistema.



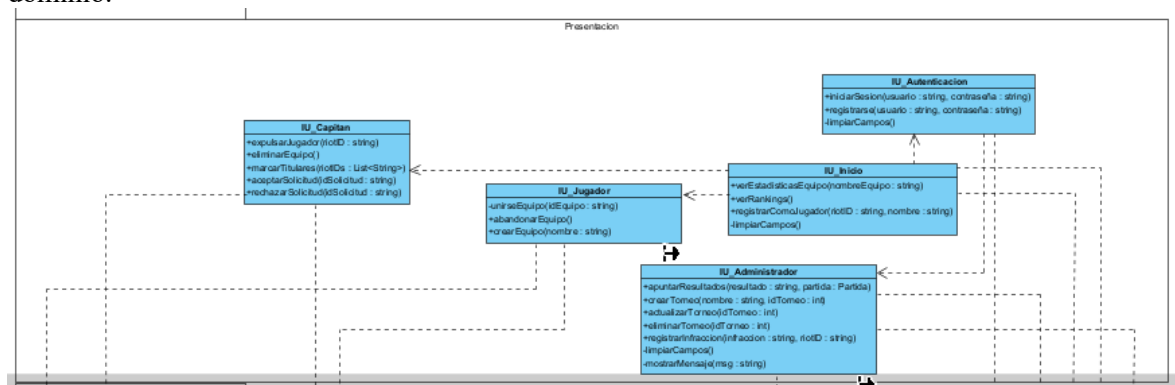
4.2 Vista estática.

4.2.1. Diagrama de clases

El diagrama de clases de diseño del sistema es el siguiente:

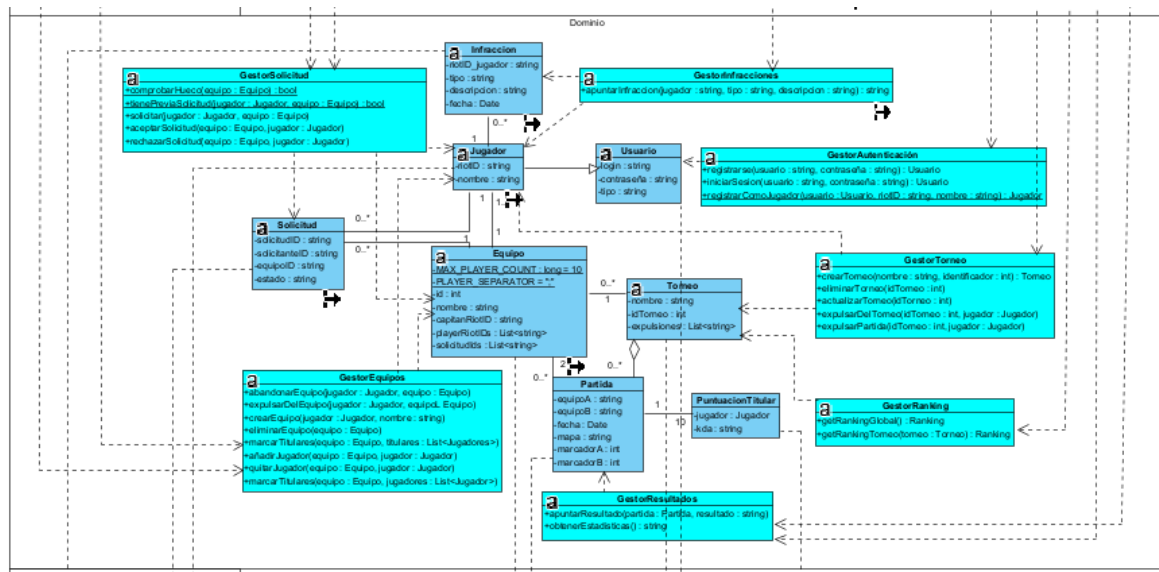
Capa presentación

La capa de presentación agrupa las clases encargadas de la interacción directa con el usuario. En ella se incluyen las interfaces de usuario asociadas a los distintos roles del sistema (jugador, capitán y administrador), como la autenticación, la consulta de información y la ejecución de los casos de uso. Esta capa se limita a recoger datos, mostrar resultados y delegar la lógica en la capa de dominio.



- Capa de dominio

La capa de dominio contiene la lógica principal del sistema. En ella se definen las entidades que representan los conceptos clave (usuario, jugador, equipo, torneo, solicitud, infracción, partida, etc.) y las clases gestoras, responsables de implementar las reglas de negocio asociadas a los distintos casos de uso. Esta capa coordina las operaciones necesarias y asegura el cumplimiento de los requisitos funcionales del sistema.



- Capa de persistencia

La capa de persistencia se encarga del acceso y gestión de los datos almacenados en la base de datos. Está implementada mediante un agente de base de datos, siguiendo el patrón Singleton, que centraliza las operaciones de inserción, consulta, actualización y eliminación. Esta capa garantiza un acceso coherente y controlado a la información utilizada por el sistema.

